KPIT | APEX LAB

Mutex

Ensuring safe access to shared resources in multithreaded programs.

Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

Why Synchronization Is Required

When multiple threads run inside the same process, they share memory:

- Global variables
- Static variables
- Heap memory
- File descriptors
- Peripheral buffers (UART, CAN, SPI)

Because of this, two or more threads may try to access or modify the same shared data simultaneously.

Typical problems that occur without synchronization

Race Conditions	Two threads modifying the same variable at the same time → unpredictable output.
Data Corruption	Concurrent writes to shared buffers (e.g., ring buffers, sensor logs) can corrupt data.
Inconsistent State	Thread A updates part of a structure while Thread B reads it → B gets half-updated data.
Deadlocks & Priority Inversion	Deadlock: two threads waiting on each other forever. Priority Inversion: low-priority thread holds mutex needed by high-priority thread.

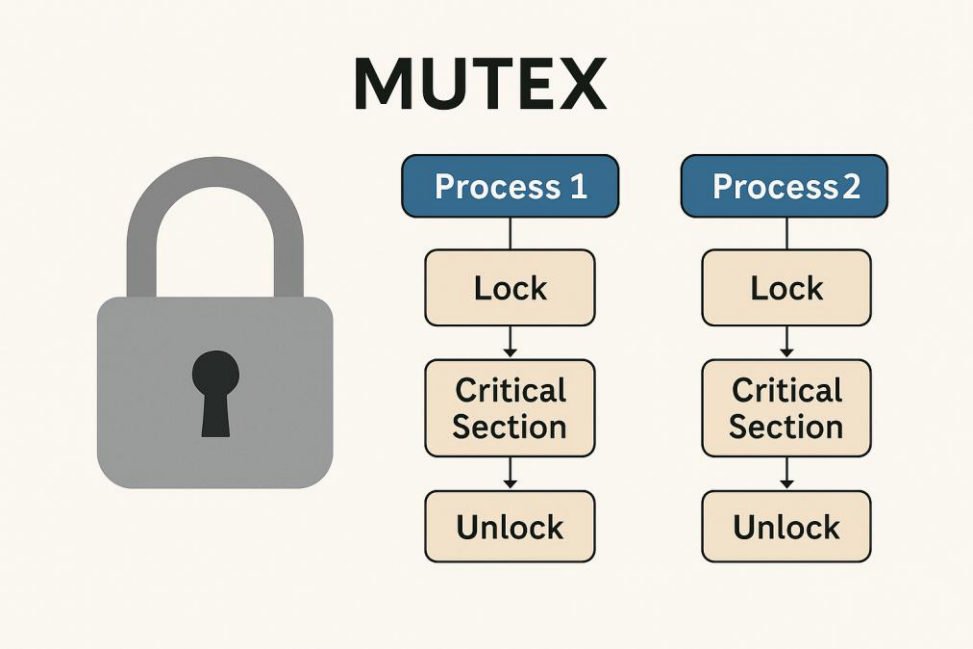
To prevent these problems, we need a mechanism that ensures only one thread at a time can access a shared resource → MUTEX

Mutex

A mutex (**MUT**ual **EX**clusion object) is a synchronization primitive that allows only one thread to own or lock a resource at a time.

It acts like:

- A key that only one thread can hold.
- Others must wait until the thread releases it.



- Mutex has two states: unlocked and locked
- Only one thread can own the mutex at a time.
- Blocking may put thread in waiting queue managed by the kernel.

How a Mutex Works

State of a Mutex

Basic Operations

1.pthread_mutex_lock()

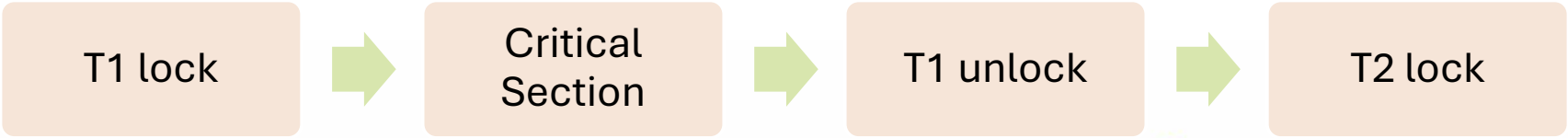
1. If unlocked → lock it and continue.
2. If locked → thread goes into **blocked state**.

2.pthread_mutex_unlock()

1. Releases mutex so another waiting thread can acquire it.

3.pthread_mutex_trylock()

1. Attempts lock without blocking.



When to Use a Mutex

Shared data must be modified

Updating a global counter
Writing to shared sensor buffers
Modifying linked lists, queues, ring buffers
Accessing shared hardware (I2C/SPI/UART write sequences)

Only one thread must access a critical section at a time

- Logging to a file
- Driving a motor control output
- Updating PWM values
- Shared communication stack (TCP/IP, CAN, UART Rx/Tx queues)

Protecting non-atomic operations

Even simple operations like counter++ need mutex protection if accessed by multiple threads.

Sample Example - Mutex

```c
#include <pthread.h>
#include <stdio.h>

int shared_counter = 0;
pthread_mutex_t lock;

void* worker(void *arg)
{
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        shared_counter++;    // safe update
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

```c
int main()
{
    pthread_t t1, t2;
    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);


    printf("Final counter value: %d\n", shared_counter);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

This example increments shared counter; without mutex you would get incorrect final value.

Best Practices

Minimize time spent inside mutex	Keep critical sections small and fast.
Avoid taking multiple mutexes in different orders	Prevent deadlocks.
Use priority-inheritance mutexes for real-time tasks	Avoids priority inversion issues.
Do not lock in high interrupt frequency handlers	Better use lock-free or atomic operations.
Always unlock before returning or on error paths	Use cleanup handlers if needed.

Scenario: Robot Battery Monitor

- Imagine you are developing basic software for a home-cleaning robot. The robot continuously moves around the house and its battery slowly drains. When the battery becomes too low, the robot must automatically go to the charging dock and start charging.
- To simulate this behavior, you must write a C program that uses:
  - **One background thread** → simulates the battery draining
  - **Main thread** → checks battery level periodically
  - **fork() + exec()** → to run an external "charging program" (such as your own charger program that prints *"Robot is now charging…"*)
- **Problem Statement:** Design and implement a C program that simulates a robot's battery monitoring system. The program must meet the following requirements:

Part 1: Battery Drain Simulation (Thread)

Create a thread named **battery_thread** which:

- Starts with battery_level = 100%.
- Every 1 second:
  - Reduces the battery by a random value between **1% and 10%**.
  - Prints: [Sensor] Battery = XX%
- When battery level drops **below 20%**, it must set a shared flag: low_battery = 1.
- Use a mutex to safely update the shared variables.

Part 2: Main Thread Monitoring

In the main thread:

- Periodically (every 1 second) check the **low_battery** flag.
- When low_battery == 1, print: [Main] Low battery detected! Going to charging station…

Then trigger the charging process.

Part 3: Charging Action

When low battery is detected:

- Use fork() to create a child process.
- In the **child process**, run an external program such as:
  - **Your own program** ./charger

    which prints: Robot is now charging...
- In the **parent process**:
  - Wait for the child to finish using wait().
  - Print: [Main] Charging completed!

Part 4: Reset & Continue

After charging finishes:

- Reset battery_level back to **100%**.
- Reset the flag low_battery = 0.
- Continue monitoring forever.

Mutex vs Spinlock

- Mutex: may block the thread → context switch → good for long critical sections.
- Spinlock: busy-waits → good for extremely short sections, kernel or SMP.
- User-space: Use mutex, not spinlocks.

Common Mistakes

- Forgetting to unlock on all return paths
- Double-locking the same mutex in the same thread → deadlock
- Unlocking a mutex not owned by the thread → undefined behavior
- Holding mutex while calling sleep(), blocking I/O
- Using printf inside mutex (causes heavy locking around stdio internals)